

Web application Request-Response Life cycle

"Let Me Explain What Really Happens Behind the Scenes When You Click a Button on a Website"

Today we are following a single request through the entire architecture of a modern web application, so you can understand exactly what happens from DNS resolution to backend databases.

DNS Resolution: Translating Domains into IP Addresses

When a user types example.com into their browser, the DNS acts as the internet's phone book, translating the domain into an IP address. This IP address could point to a load balancer or an AWS CloudFront distribution depending on the architecture. Quick DNS resolution is crucial because it helps reduce initial load times, ensuring users can access your site quickly. If you're using AWS Route 53, it helps direct the user to the closest available server, ensuring fast and efficient routing.

CloudFront: Caching and Content Delivery for Fast Access

Next up is the Content Delivery Network (CDN). Services like AWS CloudFront cache static assets, images, videos, and CSS files at Edge locations closer to your users. This helps reduce latency and speeds up load times, especially for users spread out across different regions. Not only does CloudFront improve speed, but it also helps reduce costs by offloading requests from your main server, especially when handling global traffic.

Web Application Firewall (WAF): Protecting Against Attacks

Before the request reaches your backend services, it passes through a Web Application Firewall (WAF). A WAF is your application's first line of defense. Imagine someone trying to hack into your system using SQL injection or cross-site scripting(XSS) attacks - WAF stops those attacks in their tracks by filtering out malicious traffic before it even hits your backend services.

NGINX: Reverse Proxy and Dynamic Content Caching

After passing through WAF, the request reaches NGINX, which serves as a reverse proxy. It routes traffic, handles SSL termination, and caches dynamic content. While NGINX offers fine-grained control, services like CloudFront or ALB can handle many of these tasks, making NGINX an optional component in certain architectures. However, when you need more control over routing, caching, or SSL termination, NGINX becomes a valuable addition.

Load Balancer: Distributing Traffic for High Availability

The load balancer ensures availability by distributing incoming traffic across multiple backend servers. With NLB (Network Load Balancer), you get low latency, high throughput handling for applications that need extreme performance, like gaming or financial services. On the other hand, an ALB (Application Load Balancer) gives you advanced routing capabilities ideal for web apps and microservices where requests need to be routed based on specific paths or hostnames. Both Network Load Balancer and Application Load Balancer operate at different layers of the OSI model.

NLB operates at the transport layer (Layer 4) of the OSI model. This means it forwards traffic based on the IP address and port number without inspecting the content of the request (such as HTTP headers or paths). It is designed to handle protocols like TCP (Transmission Control Protocol) and UDP (User Datagram Protocol), making it suitable for high-performance, low-latency applications handling millions of requests per second. This makes it ideal for real-time applications such as gaming or services where speed and throughput are critical.

ALB operates at the application layer (Layer 7) of the OSI model, which means it can inspect the content of HTTP or HTTPS requests, making it suitable for web applications, REST APIs, or microservices architecture. It can route traffic based on attributes like request paths (for example, routing based on specific API paths like /api/users or /api/products) or hostnames (for example, a request to example.com/api can be routed to a backend API service, or a request to example.com/static can be routed to a different service that handles static content).

API Gateway: Validating and Routing Requests

The API Gateway acts like a traffic cop, validating requests, ensuring users are authorized, and enforcing rate limits. Rate limiting and throttling prevent any single client from overwhelming your system, keeping everything running smoothly. After this, the request is routed to the right backend service.

Backend Services: Microservices Handling User Requests

A backend service can be running on an EC2 instance, an AWS Lambda function, or a microservice running in Kubernetes. Once the API Gateway does its job, it's up to the backend services to handle the heavy lifting. In microservices architecture, each service is dedicated to a specific function, allowing teams to work independently, scale services more easily, and make updates without impacting the entire system. For instance, one service might handle user authentication while another processes payments, with each service able to scale up or down separately while interacting with databases like PostgreSQL or DynamoDB.

Database Layer: Choosing Between SQL and NoSQL Databases

Databases are the heart of an application where your persistent data is stored. For distributed applications that need to scale dynamically, a NoSQL database like DynamoDB works best due to its flexibility and ability to handle unstructured data. Meanwhile, for applications with structured relational data, an SQL database like PostgreSQL is often the best choice. For asynchronous tasks, requests might be passed to message queues like Kafka or SQS, which handle tasks that don't need immediate processing, such as email notifications or background jobs.

Additional Services: Authentication, Caching, and Monitoring

In addition to handling requests, your application likely uses additional layers like authentication using OAuth 2.0, OpenID Connect, IAM, or JWT to verify users. Caching can be used to speed up responses, and logging tracks system health. Services like Redis or Memcached can cache frequently requested data, reducing the load on backend services. Monitoring tools like Prometheus or AWS CloudWatch ensure your system is running smoothly by tracking metrics and identifying issues before they escalate.

Response: Way back to the user

Now that your backend services and additional components have done their job, it's time for the response to make its way back to the user. This journey is just as important as the request itself and follows a similar path back through the system. The processed response is sent back to the API Gateway, which may apply additional formatting or transformations before routing it to the client. The load balancer efficiently routes the response back to the user's original request, balancing any outgoing traffic if necessary. If any static content was part of the response, such as images or cached data, it can be served directly from a CDN Edge location to minimize latency, ensuring a quicker response.

Conclusion: Delivering a Fast Response and Reliable User Experience

Finally, the response reaches the user's browser or mobile app, displaying the requested data and completing the full request-response cycle. In a well-designed system, this entire journey from the user's request to the final response happens in milliseconds, ensuring fast, reliable performance. From DNS resolution to the database and everything in between, every layer plays a crucial role in delivering a smooth, fast experience to users. Understanding both the request and response lifecycle gives you a solid foundation for designing resilient, scalable systems.